Лабораторна робота №3

Функціональне програмування

Завдання 3. Слово, унікальне для першого речення

Левківський Євгеній Сергійович

ІПС-41

1. Мета роботи

Створити програму обробки тексту підручника з програмування на Haskell, використовуючи типи для символу, слова, речення та розділового знака. Реалізувати читання з файла й нормалізацію пробільних символів.

2. Постановка завдання

Обрано третє завдання: «Знайти таке слово в першому реченні, якого немає в жодній з інших речень». Потрібно перевіряти відсутність у всьому залишку тексту. Програма виводить перше відповідне слово, а також усі можливі відповіді без повторень.

Вхідний файл містить UTF-8 текст підручника; на початку допускається BOM. При формуванні результату табуляції та послідовності пробільних символів замінюються одним пробілом. Підготовлений навчальний текст про Haskell міститься в lab3/examples/textbook.txt.

3. Типи та функції

Character є обгорткою над Char, WordToken — над Text, Punctuation — над Character. Element розрізняє слово та розділовий знак. Sentence зберігає нормалізований текст і елементи, Document — список речень.

```
newtype Character = Character Char
newtype WordToken = WordToken Text
newtype Punctuation = Punctuation Character
data Element = WordElement WordToken
             | PunctuationElement Punctuation
data Sentence = Sentence
  { sentenceText :: Text, elements :: [Element] }
newtype Document = Document [Sentence]
```

Основні функції: normalizeSpaces, parseDocument, sentenceWords, analyze, renderAnalysis. Вони чисті: результат залежить від аргументів. Читання, запис та повідомлення про помилки відокремлено в Main.hs, що дозволяє тестувати алгоритм без файлової системи.

4. Нормалізація і розбір

T.unwords . T.words замінює пробільні послідовності одним пробілом. Межами речень є крапка, знак питання та знак оклику. Послідовність ?! або ... утворює одну межу; завершальні лапки та дужки приєднуються до речення. Крапка між цифрами в 3.14 не завершує речення. Останній фрагмент без завершального знака враховується.

Слово починається з літери Unicode. Апостроф і дефіс входять до слова, якщо розташовані між літерами. ToCaseFold усуває різницю регістру, апострофи ’ та ’ зводяться до однакової форми. Отже, «Пам’ять» та «пам'ять» порівнюються як одне слово.

Лематизація не виконується: «функція» й «функцію» є різними словами. Граматичні скорочення на кшталт «т.д.» окремо не розпізнаються. Ці правила явно визначають, як інтерпретується текст.

5. Алгоритм

Якщо документ порожній, результат порожній. Інакше відокремлюємо перше речення. Слова решти речень збираються в Set WordToken. Потім слова першого речення переглядаються в початковому порядку.

Слово додається до відповіді, якщо його немає в множині решти тексту та ще немає серед знайдених. Друга множина усуває повторення, а накопичення у зворотному списку з подальшим reverse зберігає початковий порядок.

Наприклад, у тексті «Код тип слово. Тип. Код.» підходить лише «слово». Перевірка тільки другого речення дала б неправильну відповідь «код», адже воно присутнє в третьому. Алгоритм ураховує всі наступні речення.

Якщо текст містить лише одне речення, підходять усі його різні слова. Якщо кожне слово першого повторюється далі, виводиться повідомлення про відсутність відповіді. Для першого речення без слів відповідей також немає.

Токенізація потребує O(C) операцій за кількістю символів C. Побудова множин і перевірка членства — O(W log V), де W — кількість слів, V — кількість різних слів. Використовуються пам’ять для тексту та множин.

6. Запуск і результат

```
lab3 lab3/examples/textbook.txt
lab3 lab3/examples/textbook.txt --output lab3/results.txt
```

На Windows виконуваний файл розміщено в .build/lab3.exe. Результат друкується в консолі й за потреби зберігається як UTF-8. Недоступний файл або некоректний UTF-8 дає повідомлення й ненульовий код завершення.

```
Речень: 11
Слово першого речення, якого немає в інших: функціональна
Усі відповідні слова: функціональна, програмування
```

Перше речення: «Haskell — функціональна мова програмування.» Слова Haskell і «мова» трапляються далі. «Функціональна» та «програмування» відсутні в усіх інших реченнях. Перша відповідь — «функціональна», тому що порядок слів збережено.

Повний нормалізований текст та вивід включено до lab3/results.txt, журнал запуску — до verification/lab3.txt.

7. Перевірки

Умова	**Очікувана поведінка**
Порожній текст	Відповіді немає
Одне речення	Підходять усі різні слова
Усі слова повторюються далі	Відповіді немає
Повторення в першому реченні	У результаті одне входження
Різний регістр і апострофи	Порівняння після нормалізації
Слово лише в третьому реченні	Виключається з відповіді
Декілька допустимих відповідей	Початковий порядок збережено
Табуляції, 3.14, ?!	Коректна нормалізація й межі

Усі перевірки пройшли. Сукупний набір трьох лабораторних містить 44 автоматичні тести. Інтеграційний запуск окремо підтвердив читання реального файла та збереження результату.

8. Висновок

Виконано третє завдання: знайдено слово першого речення, відсутнє в кожному з інших. Використано явну типову модель тексту, множини, Unicode-нормалізацію та чисті функції. Для навчального прикладу відповідь — «функціональна»; також підходить «програмування».

9. Джерела

GHC User’s Guide: https://downloads.haskell.org/ghc/9.6.7/docs/users_guide/

Haskell text та containers: https://hackage.haskell.org/